

Desenvolvimento em Linux Embarcado (Raspberry Pi)

A **Raspberry Pi** têm o mesmo **pinout** e **barramentos** (GPIO, I²C, SPI, UART) em TODAS as versões, mudando apenas em desempenho e ajustes de configuração.

Componentes Necessários

- **Raspberry Pi (qualquer versão)**
Todas funcionam para fins de desenvolvimento embarcado.
 - **Cartão microSD (mínimo 16 GB ou 8GB, recomendado 32 GB)**
Espaço suficiente para o sistema, ferramentas e dependências.
 - No caso de se usar um cartão SD de 8GB, vai ser necessário ficar mais atento ao armazenamento na placa, pois com a instalação do sistema (Rasp OS Lite), alguns programas como o neovim, além da instalação das dependências do projeto, pode acabar ficando pesado demais, dificultando o desenvolvimento.
-

Componentes Opcionais

- **Componentes eletrônicos externos**
Para testar GPIO, I²C, SPI, etc.
 - **Cabo de rede (Ethernet)**
Garante conexão estável e facilita configurações via RJ45.
 - **Dispositivos USB**
Úteis para testar alimentação e comunicação USB do Linux, além de facilitar o estudo de entrada e saída no Linux.
-

Com esses itens, o ambiente estará pronto para instalação do sistema, testes de hardware e desenvolvimento embarcado.

Configuração e Preparação do Ambiente Linux:

<https://www.raspberrypi.com/software>, através desse link, baixar o Raspberry Pi Imager, software necessário para poder instalar o sistema no cartão SD, tornando o bootável.

Raspberry Pi Imager

Raspberry Pi Imager is the quick and easy way to install **Raspberry Pi OS** and other operating systems to a microSD card, ready to use with your Raspberry Pi.

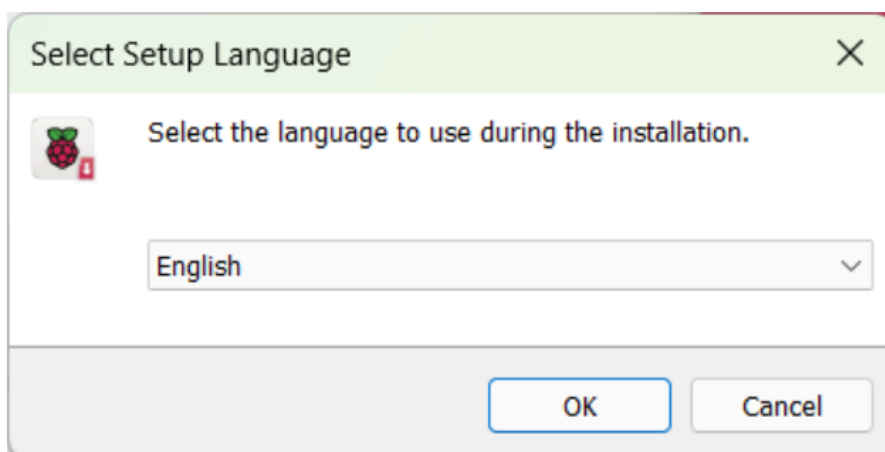
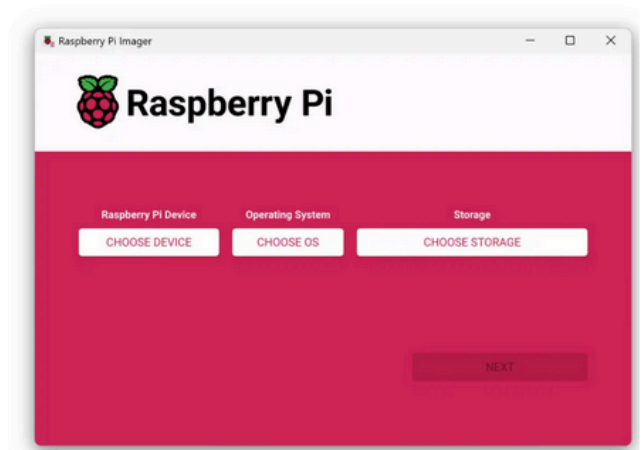
Download and install Raspberry Pi Imager on a computer with an SD card reader. Insert the microSD card you'll use with your Raspberry Pi into the reader and run Raspberry Pi Imager.

[Download for Windows](#)

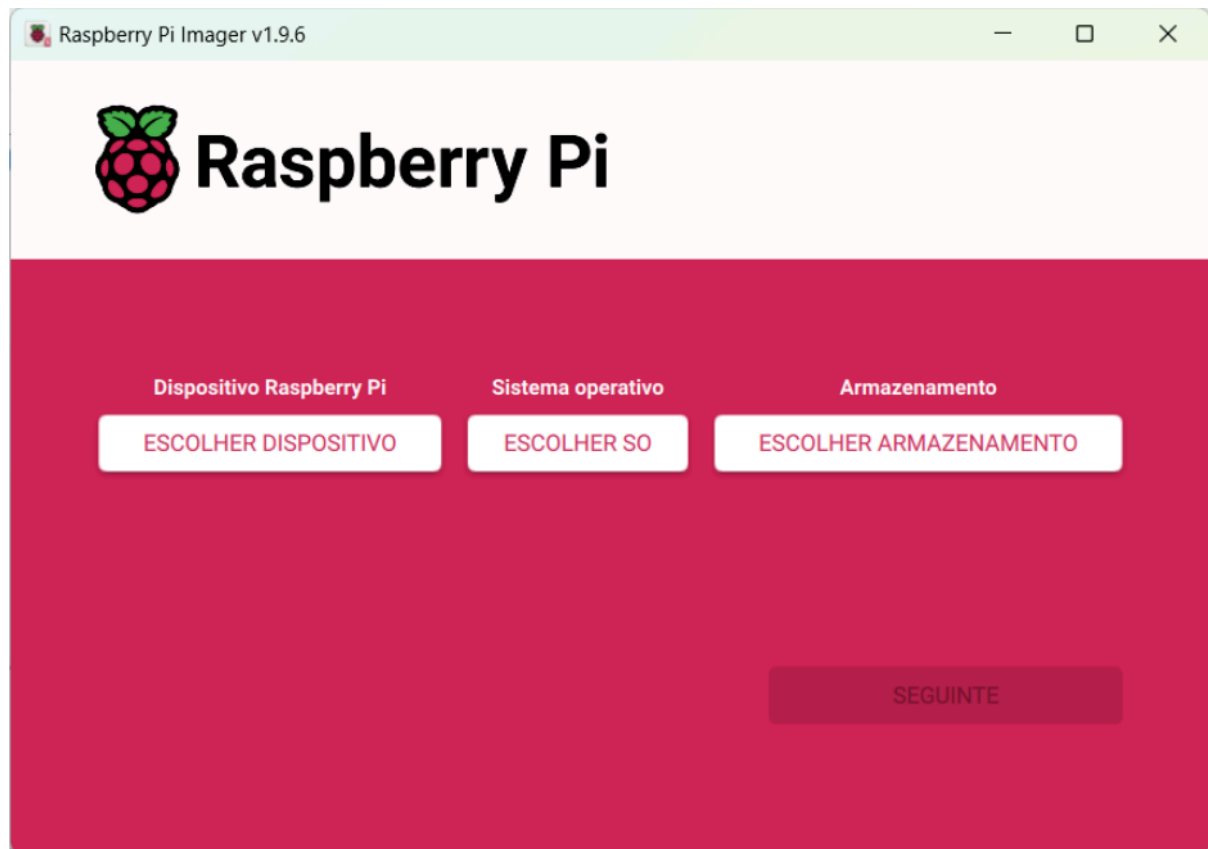
[Download for macOS](#)

[Download for Debian or Ubuntu \(x86_64\)](#)

To install on **Raspberry Pi OS**, type
`sudo apt install rpi-imager`
into a terminal window

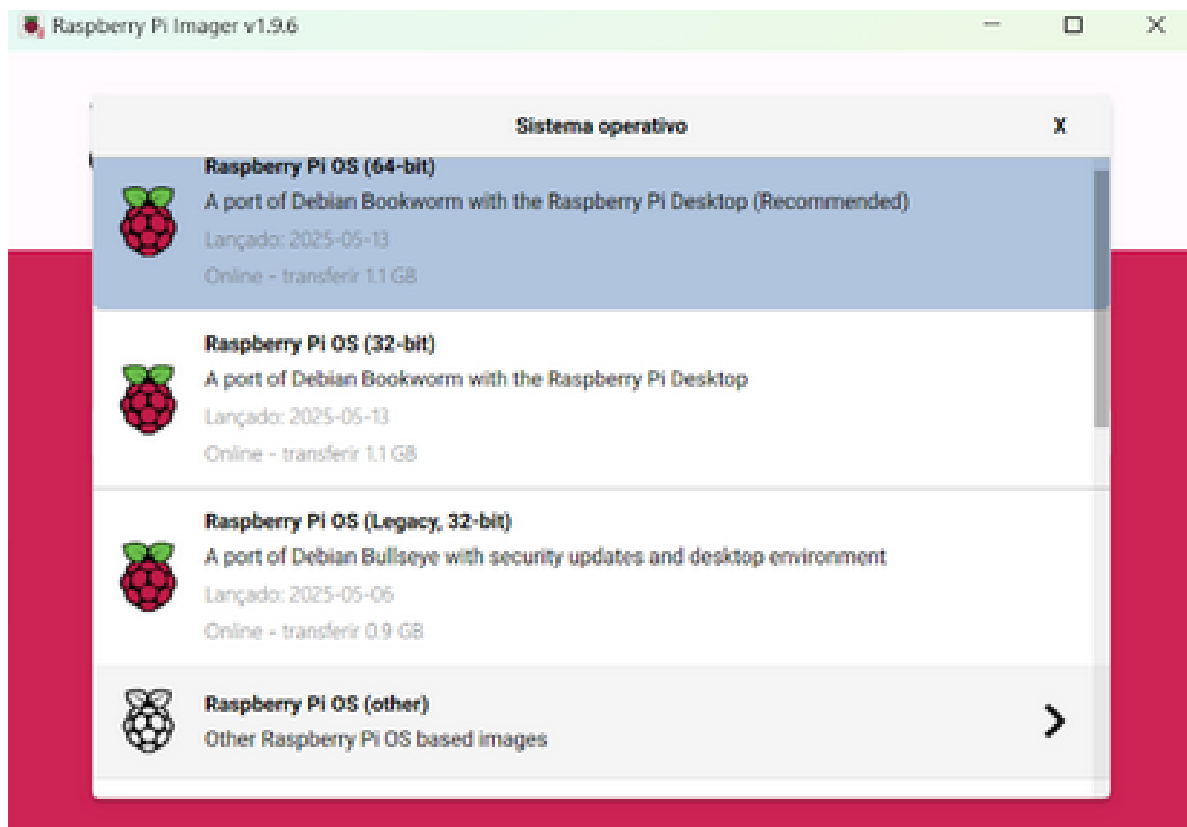


Siga o processo de instalação normalmente, dizendo OK para tudo, não há nenhuma nuância na instalação desse software, apenas instale tudo e rode o mesmo para começar a configuração real da placa.

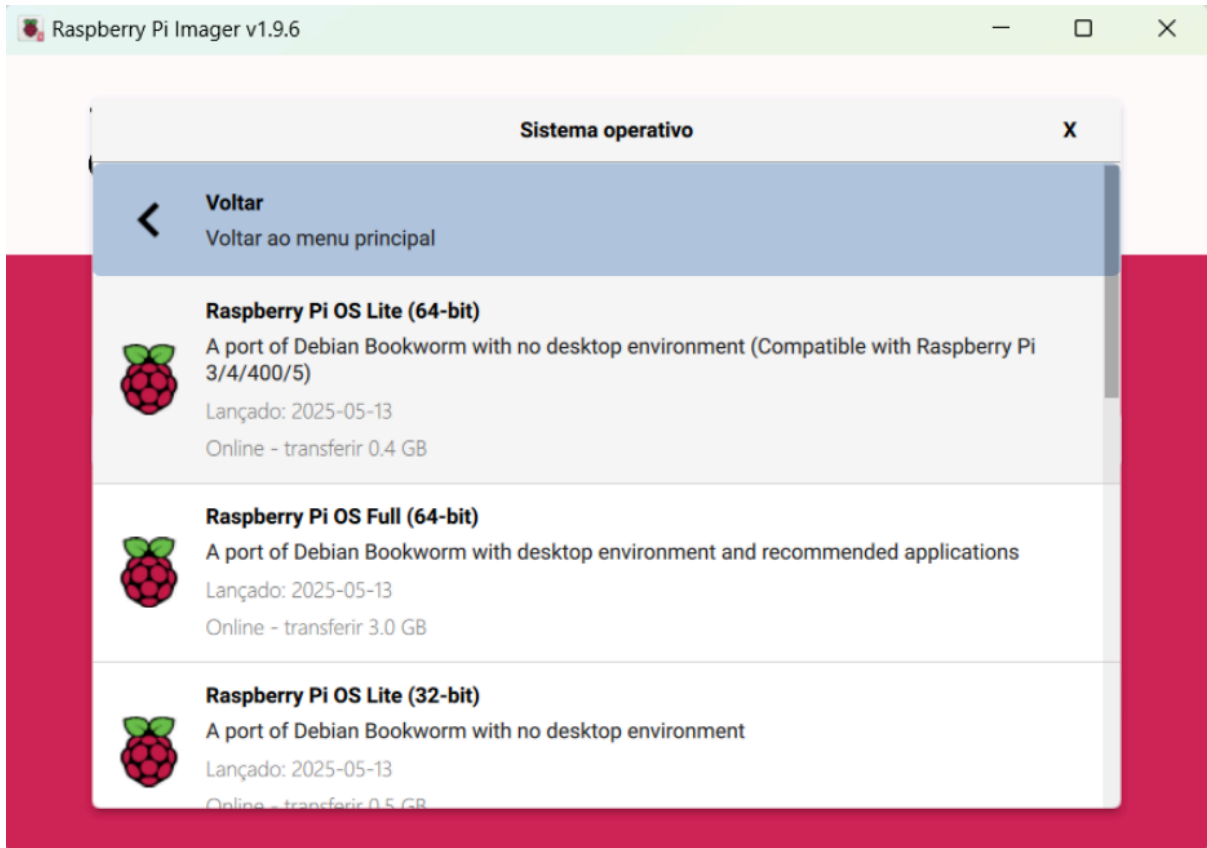


Selecione o cartão SD em “Escolher Armazenamento”, o mesmo será formatado para a instalação do sistema, naturalmente, na Escolha do SO, será necessário procurar pelo Raspberry OS Lite.

- Primeiro entre na Aba “Raspberry Pi OS (other):

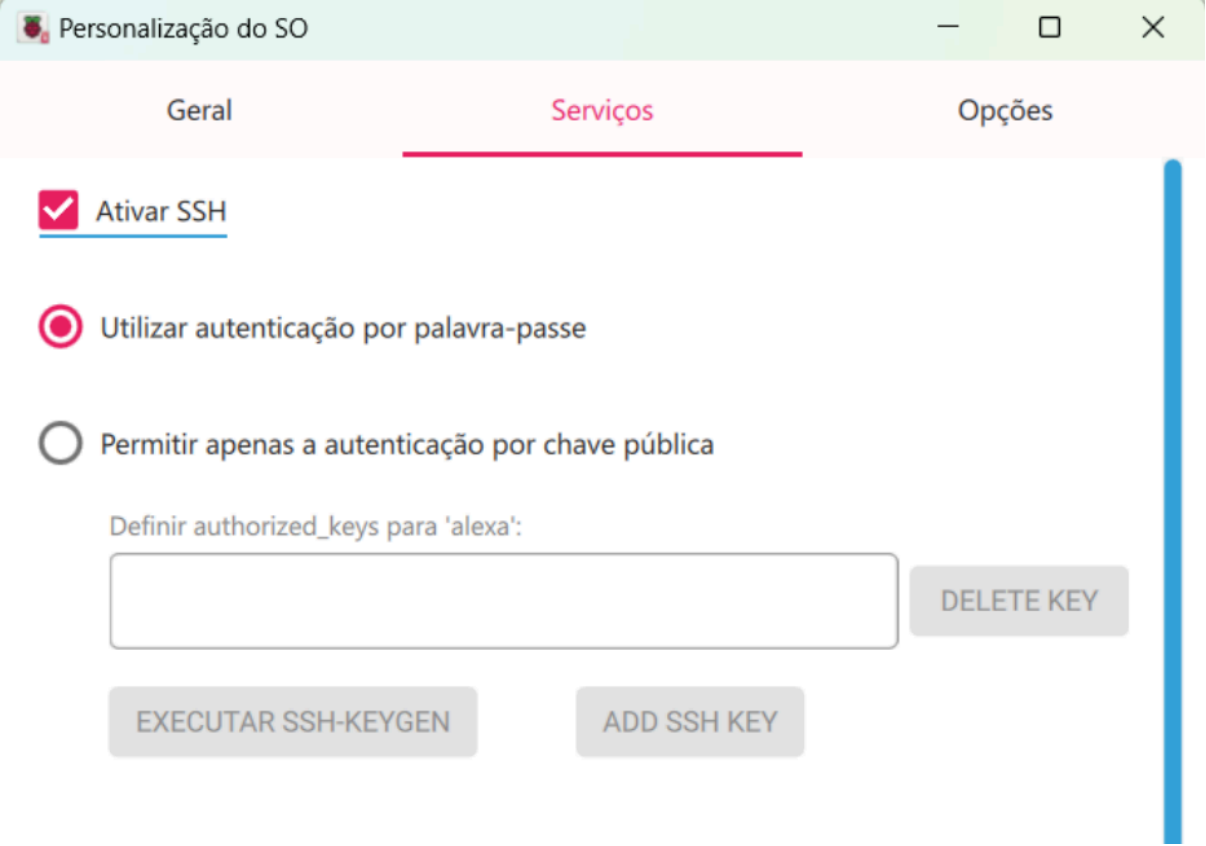


- Depois clique na primeira opção: “Raspberry Pi OS Lite (64-bit)”



- Ao clicar em next, você será questionado sobre personalização do SO, onde você irá definir o nome de usuário e senha, além da rede que ele irá iniciar conectado, é importante fazer isso para facilitar a configuração na Raspberry
- 1.a. Na janela “**Use OS customisation?**” clique em “**EDIT SETTINGS**” que irá abrir a janela “**OS Customisation**” e edite os seguinte campos na aba “**GENERAL**”:
 - **Set hostname** - escolha um nome para seu dispositivo
 - Em “**Set username and password**” configure um nome e senha para o usuário do dispositivo.
 - Em “**Configure wireless LAN**” configure os dados da rede WiFi que será utilizado para conectar o dispositivo.
 - **SSID** - Nome da rede que será utilizada para conectar o dispositivo.
 - **Password** - Senha da rede que irá ser conectada
 - **Wireless LAN country** - Definir para “BR”

- 1.b. Na aba “**SERVICES**” configure os serviços que irão ser ativados na instalação do SO.
- Marque “**Enable SSH**”.
 - E selecione a opção “**Use password authentication**”



The screenshot shows a window titled "Personalização do SO" with three tabs: "Geral", "Serviços", and "Opções". The "Serviços" tab is selected. Under this tab, there are three radio button options. The first option, "Ativar SSH", is checked with a red checkmark. The second option, "Utilizar autenticação por palavra-passe", is selected with a red circle. The third option, "Permitir apenas a autenticação por chave pública", is unselected. Below these options, there is a text input field labeled "Definir authorized_keys para 'alexa':" and a "DELETE KEY" button. At the bottom, there are two buttons: "EXECUTAR SSH-KEYGEN" and "ADD SSH KEY". A blue vertical bar is visible on the right side of the window.

Personalização do SO

Geral **Serviços** Opções

☒ Ativar SSH

☒ Utilizar autenticação por palavra-passe

☐ Permitir apenas a autenticação por chave pública

Definir authorized_keys para 'alexa':

DELETE KEY

EXECUTAR SSH-KEYGEN ADD SSH KEY

- Finalize a configuração e depois faça a confirmação da configuração para poder gravar na placa
 - Após algum tempo, se tudo tiver sido corretamente configurado, o cartão SD será ejetado automaticamente e poderá ser colocado na Raspberry PI.

Device Tree na Raspberry Pi

O **Device Tree (DT)** é uma tecnologia crucial no ecossistema Linux embarcado, atuando como um mecanismo para **separar a descrição do hardware do código-fonte do kernel**. Isso é fundamental para a **portabilidade** do kernel, permitindo que o mesmo binário do Linux seja executado em diversas placas Raspberry Pi que possuem variações de hardware.

O que é e Como Funciona

O Device Tree é, essencialmente, uma estrutura de dados hierárquica (uma árvore) que descreve o hardware do sistema de forma declarativa. Ele lista:

- **Periféricos** (GPIOs, SPI, I²C, UART).
- **Barramentos e controladores**.
- **Propriedades** de cada dispositivo (endereço de memória, IRQ - Interrupt Request, clock).

Essa descrição é fornecida ao kernel durante o boot. O kernel, então, usa essa informação para:

1. **Identificar** os dispositivos disponíveis.
2. **Alocar recursos** (memória, interrupções).
3. **Carregar os drivers** apropriados que "entendem" os nós do Device Tree (mecanismo conhecido como *Device Tree Binding*).

Componentes Chave na Raspberry Pi

O DT é implementado através de três tipos de arquivos:

1. **Device Tree Source (.dts)**: Arquivo de texto que contém a descrição completa do hardware.
2. **Device Tree Blob (.dtb)**: A versão compilada e binária do **.dts**, armazenada em **/boot/** (ou **/boot/firmware/**) e carregada pelo bootloader. O Raspberry Pi carrega o **.dtb** correspondente ao seu modelo.
3. **Device Tree Overlays (.dtbo)**: Arquivos complementares usados para **modificar ou adicionar nós** à árvore principal em tempo de boot. Eles são cruciais para hardware opcional, como HATs ou módulos específicos (Ex: **i2c-rtc.dtbo** para adicionar um relógio de tempo real I²C).

Configuração e o Arquivo **config.txt**

Na Raspberry Pi, o arquivo **/boot/config.txt** atua como a interface principal para manipulação do DT:

- **dtoverlay=**: Usado para carregar um ou mais arquivos `.dtbo` (overlays).
- **dtparam=**: Usado para passar parâmetros para um nó existente no DT ou em um overlay. O exemplo `dtparam=act_led_trigger=heartbeat` é uma maneira de alterar uma propriedade do nó do LED de atividade sem recompilar o kernel, usando um parâmetro exposto pelo próprio DT.

Interação em Tempo de Execução via sysfs

Após o kernel ler o DT e inicializar os drivers, os dispositivos são expostos ao espaço do usuário através do sistema de arquivos virtual **sysfs** (Sistema de Arquivos do Sistema), localizado em `/sys`.

Para LEDs, por exemplo, o caminho é `/sys/class/leds/<nome_do_led>/`.

- **trigger**: O arquivo `trigger` permite alterar o modo de operação do LED dinamicamente (Ex: `heartbeat`, `timer`, `default-on`, `none`).
- **brightness**: Permite ligar (1) ou desligar (0) o LED quando o `trigger` está definido como `none` (controle manual).

Essa interface **sysfs** é a forma padronizada do Linux de interagir com hardware, aproveitando as descrições fornecidas pelo Device Tree.

GPIO na Raspberry Pi

O curso foca na **manipulação avançada dos pinos GPIO** (General Purpose Input/Output) da Raspberry Pi dentro do contexto do **Linux Embarcado**. O objetivo é capacitar o desenvolvedor a escolher e aplicar o método de controle de hardware mais eficiente e adequado para cada tipo de projeto, desde protótipos rápidos até sistemas de tempo real.

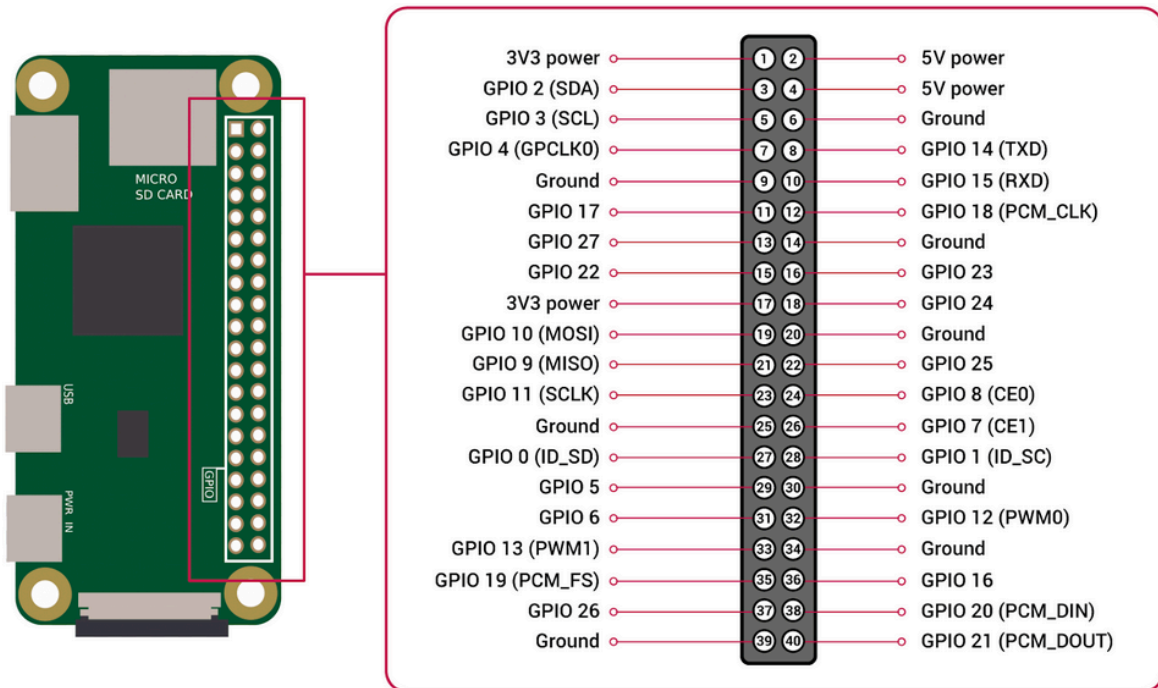
I. Acesso ao Hardware e o Kernel Linux

A Raspberry Pi utiliza o **Linux Embarcado (Raspberry Pi OS)**, que atua como mediador entre a aplicação e o hardware físico. O acesso aos pinos de 3,3 V é dividido em três camadas:

1. **Camada de Kernel (Drivers)**: É a forma mais robusta de acesso. O Linux moderno substituiu a interface antiga `/sys/class/gpio/` (sysfs) pela interface **Character Device** (cdev) exposta em `/dev/gpiochipN`. A biblioteca **libgpiod** é o padrão oficial do kernel para interagir com essa nova interface, garantindo estabilidade e aderência aos padrões do Linux.
2. **Camada de Bibliotecas (User-Space)**: Bibliotecas como **pigpio**, **RPi.GPIO** e principalmente **lgpiod** (ou *wrappers* mais recentes como **gpiozero** e **periphery**)

forneem APIs de alto nível. Elas simplificam a programação, mas podem introduzir uma pequena latência, pois não estão diretamente no kernel.

3. **Camada de Aplicação:** Uso de linguagens de script (Python) ou utilitários (Shell Script) que chamam as bibliotecas de *user-space*.



II. O Padrão libgpiod (Método Moderno)

A **libgpiod** é o framework recomendado para controle de GPIO em todas as distribuições Linux modernas, incluindo a Raspberry Pi OS. Ele resolve os problemas de *race conditions* e gerenciamento de recursos da interface sysfs descontinuada.

Ferramenta
(gpiod-tools)

Descrição e Aplicação

gpioset

Usado para **configurar o modo (saída/entrada)** e **escrever valores** nas linhas GPIO. Permite variações como *open-drain*, *open-source* e segurar o estado por um tempo (`--mode=time`). Ideal para scripts de *boot* e testes rápidos de pinos.

gpioget	Usado para ler o valor lógico (0 ou 1) de uma linha GPIO. Útil para verificar o estado de sensores e botões.
gpioinfo	Lista todos os chips de GPIO (controladores) disponíveis no sistema e as linhas (offsets BCM) que eles controlam, incluindo suas funções.
gpiofind	Localiza a linha GPIO pelo nome simbólico do <i>Device Tree</i> , facilitando a portabilidade do script.

III. Comparativo de Linguagens para o Controle de Hardware

A escolha da linguagem define o **desempenho**, a **produtividade** e o nível de **controle de baixo nível** da aplicação embarcada.

1. Shell Script (Bash)

- **Vantagens:** Rapidez no desenvolvimento, comandos nativos, ótimo para automação e sequências simples.
- **Desvantagens:** Não é adequado para timing preciso (microssegundos), exige a instalação da **libgpiod** para comandos modernos (**gpioset**).

2. C (Linguagem com acesso simples a ponteiro)

- **Vantagens:** **Desempenho máximo e determinismo** (previsibilidade de tempo). Essencial para controle de motores, protocolos seriais de alta velocidade e sistemas que não podem falhar em tempo.
- **Bibliotecas Comuns:** **libgpiod**, **pigpio** (oferece *bit-banging* e precisão em microssegundos), e **bcm2835** (acesso direto aos registradores do SoC Broadcom).
- **Desvantagens:** Curva de aprendizado acentuada e maior tempo de desenvolvimento para prototipagem.

3. Python (Linguagem de Alto Nível)

- **Vantagens:** **Alta produtividade**, vasta comunidade, sintaxe simples. Ideal para prototipagem rápida, IoT e integração com bibliotecas de Machine Learning.
 - **Bibliotecas Comuns:** **gpiozero** (mais didática e abstrata), **RPi.GPIO** (tradicional e eficaz), **periphery** (usa a interface moderna do kernel) e **I2C** (mais comum usada nas bibliotecas atuais como da Adafruit Circuit Python).
 - **Desvantagens:** **Não é determinístico** devido ao *Garbage Collection* e à natureza interpretada, o que pode causar latência em aplicações de tempo real.
-

IV. Pinagem e Referência

BCM (GPIO) ou a **BOARD (Posição Física)** tem **40 pinos**, conforme o modo configurado na biblioteca:

- **BCM:** Refere-se ao número do pino no SoC Broadcom (e.g., GPIO 18, usado no comando `gpioset`).
- **BOARD:** Refere-se à posição física no cabeçalho (e.g., Pino 12).

Interfaces comuns como I²C, SPI, UART e PWM estão disponíveis em pinos específicos, cujas funções alternativas são ativadas e configuradas via **Device Tree Overlays** (discutido na aula anterior).

I²C na Raspberry Pi

O **I²C (Inter-Integrated Circuit)** na Raspberry Pi para conectar um controlador (Mestre, no caso, a Raspberry Pi) a múltiplos periféricos (Escravos). É necessário entender os seguintes pontos:

I. Características Essenciais do I²C

- **Barramento Bipolar:** Permite múltiplos dispositivos no mesmo barramento, identificados por um **endereço único** (geralmente de 7 bits).
 - **Linhas e Tensão:** Usa as linhas SDA e SCL e opera a **3,3 V** na Raspberry Pi.
 - **Pull-up:** Requer **resistores de pull-up** externos (ou internos em alguns módulos) para garantir que as linhas fiquem em nível lógico alto quando não estiverem sendo utilizadas.
 - **Velocidade:** Suporta velocidades padrão (100 kHz) e rápidas (400 kHz, 1 MHz).
-

II. I²C na Raspberry Pi e Linux

O barramento I²C é um recurso nativo da Raspberry Pi, gerenciado pelo kernel Linux, o que facilita drasticamente a programação.

1. Barramento Principal

- O barramento I²C utilizado para uso geral é o **I²C-1**, exposto no cabeçalho GPIO:
 - **Pino 3:** GPIO 2 (SDA1)
 - **Pino 5:** GPIO 3 (SCL1)
- O barramento I²C-0 é geralmente reservado para uso interno.

2. Abstração no Linux

No Linux embarcado, o kernel gerencia a comunicação I²C e a expõe ao usuário como um arquivo de dispositivo em `/dev/`, tipicamente o `/dev/i2c-1`.

- **Fluxo de Trabalho:** O programador interage com este arquivo de dispositivo ou usa bibliotecas (como **smbus** em Python), sem precisar controlar o protocolo "bit a bit".
-

III. Configuração e Ferramentas

Para utilizar o I²C na Raspberry Pi OS, é necessário habilitar e usar ferramentas de *user-space*.

1. Habilitação

O barramento deve ser ativado no arquivo de configuração do boot (`/boot/firmware/config.txt`) adicionando ou descomentando:

dtparam=i2c_arm=on

Após a reinicialização, o dispositivo `/dev/i2c-1` deve aparecer. Caso não apareça, o módulo `i2c-dev` deve ser carregado manualmente ou de forma permanente em `/etc/modules`.

2. Pacotes e Utilitários

É essencial instalar o pacote **i2c-tools** para diagnóstico e testes rápidos:

- **i2cdetect:** Usado para **escanear o barramento** I²C e listar os endereços dos dispositivos conectados (Ex: `sudo i2cdetect -y 1`).
 - **i2cget:** Lê um valor de um registrador específico de um dispositivo escravo.
 - **i2cset:** Escreve um valor em um registrador específico de um dispositivo escravo.
 - **python3-smbus:** Biblioteca padrão em Python para interagir com o I²C.
-

UART na Raspberry Pi

I. Fundamentos da Comunicação UART

A correta comunicação exige que ambos os dispositivos concordem com os seguintes parâmetros:

- **Baud Rate:** Taxa de transmissão de bits (ex: 9600, 115200 bps).
- **Formato do Quadro:** Definido por **Start bit**, **Data bits** (7 ou 8), **Paridade** (opcional), e **Stop bits** (1 ou 2).
- **Nível Lógico:** A Raspberry Pi usa **3,3 V TTL**. Nunca conecte 5 V diretamente.

- **Controle de Fluxo (Flow Control):** Pode ser por hardware (**RTS/CTS**) ou software (**XON/XOFF**), para evitar perda de dados por buffer cheio.

II. UART no Linux Embarcado

No Linux, o controlador UART do hardware é gerenciado por um *driver* do kernel e exposto como um arquivo de dispositivo no diretório `/dev/`.

Device Node	Controlador Típico	Características	Uso Recomendado
<code>/dev/ttyAMA0</code>	PL011	UART completa (<i>full-featured</i>), clock estável, FIFO.	Ideal para periféricos que exigem precisão.
<code>/dev/ttyS0</code>	mini-UART	UART simplificada, baud rate instável (dependente do <i>core clock</i> da CPU).	Desaconselhável para alta precisão.
<code>/dev/serial0</code>	Symlink	Aponta para o UART configurado para uso no GPIO.	Usar no código para maior portabilidade entre modelos.

SPI na Raspberry Pi e Linux Embarcado

I. Fundamentos do Protocolo SPI

O SPI requer **quatro linhas de sinal** para operação:

1. **SCLK (Serial Clock):** Sinal de clock gerado pelo dispositivo mestre (Raspberry Pi).
2. **MOSI (Master Out Slave In):** Linha de dados enviada do Mestre para o Escravo.
3. **MISO (Master In Slave Out):** Linha de dados enviada do Escravo para o Mestre.
4. **CS/CE (Chip Select / Chip Enable):** Sinal de seleção. O mestre usa uma linha CS/CE separada para **selecionar qual escravo** na rede está ativo para a comunicação.

II. SPI no Linux Embarcado

O Linux gerencia o controlador SPI do hardware e o expõe ao *user-space* através da interface **spidev**.

- **Device Node:** O SPI é acessado via arquivos de dispositivo em `/dev/spidevX.Y`, onde:
 - **X** é o número do barramento SPI (ex: `0` para SPI0).
 - **Y** é o número do Chip Select (CS) ou Chip Enable (CE) usado (ex: `0` para CE0, `1` para CE1).
 - **Exemplo:** `/dev/spidev0.0` (Barramento SPI0, Chip Select 0).
- **Programação:** A comunicação é implementada através de bibliotecas (como `python3-spidev`) que interagem com o arquivo `spidev`.

III. Configuração na Raspberry Pi

A Raspberry Pi (modelos como Zero 2W e 3B) expõe o barramento **SPI0** no cabeçalho de 40 pinos, operando a **3,3 V**.

1. Pinos Padrão do SPI0

- **Pino 19 (GPIO 10):** MOSI
- **Pino 21 (GPIO 9):** MISO
- **Pino 23 (GPIO 11):** SCLK
- **Pino 24 (GPIO 8):** CE0
- **Pino 26 (GPIO 7):** CE1

2. Habilitação no Boot

Para habilitar o suporte ao SPI e criar os nós de dispositivo (`/dev/spidev*`), é necessário editar o arquivo de configuração do boot:

- **Arquivo:** `/boot/firmware/config.txt`
- **Parâmetro:** Adicionar ou descomentar a linha:

```
dtoverlay=spi=on
```

- Isso ativa o *Device Tree Overlay* necessário para o SPI.

3. Verificação e Módulos

Após reiniciar, a habilitação é confirmada pela presença dos nós de dispositivo:

Bash

```
ls /dev/spidev*
```

Resultado esperado: `/dev/spidev0.0 /dev/spidev0.1`

Se os dispositivos não aparecerem, os módulos do kernel `spi_bcm2835` e `spidev` podem ser carregados manualmente (`sudo modprobe`) e tornados permanentes em `/etc/modules`.

IV. Pacotes de Programação

Para desenvolvimento, os seguintes pacotes são usados:

- **Python:** `python3-spidev` (biblioteca Python para acessar a interface `spidev`).
- **C:** `libspi-dev` (cabeçalhos para acesso em C).